**MONOLITH**

**app.py**

```
from flask import Flask, request, jsonify
app = Flask(__name__)
cart = []
@app.get("/")
def home():
    return jsonify(
        application="Shopping App Monolith",
        services=["login", "cart", "checkout"]
    )
@app.get("/health")
def health():
    return jsonify(status="healthy", application="shopping-monolith")
@app.post("/login")
def login():
    data = request.get_json(silent=True) or {}
    if data.get("username") == "admin" and data.get("password") == "1234":
        return jsonify(message="Login successful", username="admin"), 200
    return jsonify(message="Invalid credentials"), 401
@app.get("/cart")
def get_cart():
    return jsonify(cart=cart)
@app.post("/cart")
def add_to_cart():
    data = request.get_json(silent=True) or {}
    item, price = data.get("item"), data.get("price")
    if not isinstance(item, str) or not item.strip():
        return jsonify(error="item is required"), 400
    if isinstance(price, bool) or not isinstance(price, (int, float)) or price
< 0:
        return jsonify(error="price must be a non-negative number"), 400
    product = {"item": item.strip(), "price": price}
    cart.append(product)
    return jsonify(message="Item added", product=product, cart=cart), 201
@app.post("/checkout")
def checkout():
    data = request.get_json(silent=True) or {}
    items = data.get("items")
    if not isinstance(items, list) or not items:
        return jsonify(error="items must be a non-empty list"), 400
    total = 0
    for product in items:
        if not isinstance(product, dict):
            return jsonify(error="each item must be an object"), 400
        price = product.get("price")
        if isinstance(price, bool) or not isinstance(price, (int, float)) or
price < 0:
            return jsonify(error="each item needs a valid price"), 400
        total += price
    return jsonify(message="Checkout successful", items=items,
                   total=round(total, 2), order_status="confirmed"), 200
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Dockerfile**

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

**compose.monolith.yaml**

```
services:
  shopping-monolith:
    build: ./monolith
    image: shopping-monolith:1.0
    ports:
      - "5000:5000"
```

**SERVICES----CART**

**app.py**

```
from flask import Flask, request, jsonify
app = Flask(__name__)
cart = []
@app.get("/")
def home():
    return jsonify(message="Cart Service")
@app.get("/health")
def health():
    return jsonify(status="healthy", service="cart")
@app.get("/cart")
def get_cart():
    return jsonify(cart=cart)
@app.post("/cart")
def add_to_cart():
    data = request.get_json(silent=True) or {}
    item, price = data.get("item"), data.get("price")
    if not isinstance(item, str) or not item.strip():
        return jsonify(error="item is required"), 400
    if isinstance(price, bool) or not isinstance(price, (int, float)) or price
< 0:
        return jsonify(error="price must be a non-negative number"), 400
    product = {"item": item.strip(), "price": price}
    cart.append(product)
    return jsonify(message="Item added", product=product, cart=cart), 201
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Dockerfile**

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

**CHECKOUT**

**app.py**

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.get("/")
def home():
    return jsonify(message="Checkout Service")
@app.get("/health")
def health():
    return jsonify(status="healthy", service="checkout")
@app.post("/checkout")
def checkout():
    data = request.get_json(silent=True) or {}
    items = data.get("items")
    if not isinstance(items, list) or not items:
        return jsonify(error="items must be a non-empty list"), 400
    total = 0
    for product in items:
        if not isinstance(product, dict):
            return jsonify(error="each item must be an object"), 400
        price = product.get("price")
        if isinstance(price, bool) or not isinstance(price, (int, float)) or
price < 0:
            return jsonify(error="each item needs a valid price"), 400
        total += price
    return jsonify(message="Checkout successful", items=items,
                   total=round(total, 2), order_status="confirmed"), 200
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Dockerfile**

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

**LOGIN**

**app.py**

```
from flask import Flask, request, jsonify
app = Flask(__name__)
@app.get("/")
def home():
    return jsonify(message="Login Service")
@app.get("/health")
def health():
    return jsonify(status="healthy", service="login")
@app.post("/login")
def login():
    data = request.get_json(silent=True) or {}
    if data.get("username") == "admin" and data.get("password") == "1234":
```

```
        return jsonify(message="Login successful", username="admin"), 200
    return jsonify(message="Invalid credentials"), 401
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

**Dockerfile**

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

**compose.services.yaml**

```
services:
  login:
    build: ./services/login
    image: shopping-login:1.0
    ports:
      - "5001:5000"
  cart:
    build: ./services/cart
    image: shopping-cart:1.0
    ports:
      - "5002:5000"
  checkout:
    build: ./services/checkout
    image: shopping-checkout:1.0
    ports:
      - "5003:5000"
```

**k8s**

**monolith.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: shopping-monolith
spec:
  replicas: 1
  selector:
    matchLabels:
      app: shopping-monolith
  template:
    metadata:
      labels:
        app: shopping-monolith
    spec:
      containers:
        - name: shopping-monolith
          image: shopping-monolith:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: shopping-monolith-service
spec:
  selector:
    app: shopping-monolith
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
```

**services.yaml**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: login
spec:
  replicas: 1
  selector:
    matchLabels:
      app: login
  template:
    metadata:
      labels:
        app: login
    spec:
      containers:
        - name: login
          image: shopping-login:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: login-service
spec:
  selector:
    app: login
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cart
spec:
  replicas: 1
  selector:
```

```
    matchLabels:
      app: cart
  template:
    metadata:
      labels:
        app: cart
    spec:
      containers:
        - name: cart
          image: shopping-cart:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: cart-service
spec:
  selector:
    app: cart
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: checkout
spec:
  replicas: 1
  selector:
    matchLabels:
      app: checkout
  template:
    metadata:
      labels:
        app: checkout
    spec:
      containers:
        - name: checkout
          image: shopping-checkout:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 3
            periodSeconds: 5
---
apiVersion: v1
```

```
kind: Service
metadata:
  name: checkout-service
spec:
  selector:
    app: checkout
  ports:
    - port: 5000
      targetPort: 5000
  type: ClusterIP
```

**EXECUTION**

**DEMO CRED: admin/1234**

```
python -m venv .venv
venv\Scripts\Activate.ps1
pip install -r monolith/requirements.txt
python monolith/app.py
```

**2nd terminal**

```
curl http://localhost:5000/
curl -X POST http://localhost:5000/login -H "Content-Type: application/json" -d
"{\"username\":\"admin\",\"password\":\"1234\"}"
curl -X POST http://localhost:5000/cart -H "Content-Type: application/json" -d
"{\"item\":\"Laptop\",\"price\":50000}"
curl http://localhost:5000/cart
curl -X POST http://localhost:5000/checkout -H "Content-Type: application/json"
-d "{\"items\":[{\"item\":\"Laptop\",\"price\":50000}]}"
```

**Build and run the monolith with Docker**

**Build image:** docker build -t shopping-monolith:1.0 ./monolith

**Run container:**

```
docker run -d --name shopping-monolith -p 5000:5000 shopping-monolith:1.0
```

**Check:** docker ps

```
docker logs shopping-monolith
curl http://localhost:5000/health
```

**Stop and remove:** docker rm -f shopping-monolith

**Run the monolith with Docker Compose**

```
docker compose -f compose.monolith.yaml up --build -d
docker compose -f compose.monolith.yaml ps
docker compose -f compose.monolith.yaml logs -f
```

**Stop:** docker compose -f compose.monolith.yaml down

**Run three independent services with Docker Compose**

**Build, start:** docker compose -f compose.services.yaml up --build -d

**Check status and logs:**

```
docker compose -f compose.services.yaml ps
docker compose -f compose.services.yaml logs
docker compose -f compose.services.yaml logs login
docker compose -f compose.services.yaml logs cart
docker compose -f compose.services.yaml logs checkout
```

Test login:

```
curl -X POST http://localhost:5001/login -H "Content-Type: application/json" -d
"{\"username\":\"admin\",\"password\":\"1234\"}"
```

Add and view cart items:

```
curl -X POST http://localhost:5002/cart -H "Content-Type: application/json" -d
"{\"item\":\"Laptop\",\"price\":50000}"
```

curl http://localhost:5002/cart
Checkout:
curl -X POST http://localhost:5003/checkout -H "Content-Type: application/json" -d "{\"items\":[{\"item\":\"Laptop\",\"price\":50000},{\"item\":\"Mouse\",\"price\":500}]}"
**Stop services:** docker compose -f compose.services.yaml down
**Kubernetes**
**Deploy the monolith**
**Build image:** docker build -t shopping-monolith:1.0 ./monolith
**Start Minikube and load the image:**
minikube start
minikube image load shopping-monolith:1.0
**Deploy and verify:**
kubectl apply -f k8s/monolith.yaml
kubectl get deployments
kubectl get pods
kubectl get services
kubectl rollout status deployment/shopping-monolith
**Access it:**
kubectl port-forward service/shopping-monolith-service 5000:5000
Use a second terminal to test `http://localhost:5000/health`.
**Delete:** kubectl delete -f k8s/monolith.yaml

**Deploy three services**
Build the service images:
docker build -t shopping-login:1.0 ./services/login
docker build -t shopping-cart:1.0 ./services/cart
docker build -t shopping-checkout:1.0 ./services/checkout
**Load into Minikube:**
minikube image load shopping-login:1.0
minikube image load shopping-cart:1.0
minikube image load shopping-checkout:1.0
**Deploy:**
kubectl apply -f k8s/services.yaml
kubectl get deployments
kubectl get pods
kubectl get services
**Wait for rollout:**
kubectl rollout status deployment/login
kubectl rollout status deployment/cart
kubectl rollout status deployment/checkout
**Port-forward each service in a separate terminal:**
kubectl port-forward service/login-service 5001:5000
kubectl port-forward service/cart-service 5002:5000
kubectl port-forward service/checkout-service 5003:5000
**Delete:** kubectl delete -f k8s/services.yaml
**Kubernetes troubleshooting**
kubectl get pods -o wide
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl get events --sort-by=.metadata.creationTimestamp
**Common Docker commands**
docker --version

```
docker images
docker ps
docker ps -a
docker stop CONTAINER
docker start CONTAINER
docker logs CONTAINER
docker exec -it CONTAINER sh
docker rm -f CONTAINER
docker rmi IMAGE
docker compose config
docker compose down
```

**Common Kubernetes commands**

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
kubectl apply -f FILE.yaml
kubectl get pods
kubectl get deployments
kubectl get services
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl scale deployment/cart --replicas=3
kubectl rollout undo deployment/NAME
kubectl port-forward service/SERVICE_NAME LOCAL_PORT:REMOTE_PORT
kubectl delete -f FILE.yaml
```

**B. Dockerfile – important instructions**

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

**Build and run it:** `docker build -t flaskapp:1.0 .`

```
docker run -d --name flaskapp -p 5000:5000 flaskapp:1.0
```

**Part 1 – Docker CLI, Dockerfile and Docker Compose**

**A. Docker CLI commands**

**1. Check Docker installation and status**

```
docker --version
docker info
docker version
```

**2. Download images from Docker Hub**

```
docker pull nginx
docker pull ubuntu
docker pull python:3.12-slim
docker pull postgres:16-alpine
```

**3. List images**

```
docker images
docker image ls
```

**4. Run a container**

```
docker run nginx
docker run -d nginx
docker run -d --name mynginx nginx
```

- `-d` – run in the background.

- --name – assign a container name.

**5. Port mapping**

```
docker run -d --name web -p 8080:80 nginx
```

Format: -p HOST_PORT:CONTAINER_PORT

Open http://localhost:8080 in your browser.

**7. List containers:** docker ps

**8. Stop and start containers**

```
docker stop mynginx
docker start mynginx
docker restart mynginx
```

**9. Remove containers**

```
docker stop mynginx
docker rm mynginx
docker rm -f mynginx
```

**10. Container logs**

```
docker logs mynginx
docker logs -f mynginx
docker logs --tail 50 mynginx
```

**11. Inspect a container**

```
docker inspect mynginx
docker inspect -f "{{.State.Status}}" mynginx
```

**13. Build an image**

```
docker build -t myapp:1.0 .
```

- -t – sets the image name and tag.
- . – uses the current directory as the build context.

Build using a different Dockerfile:

```
docker build -f Dockerfile.dev -t myapp:dev .
```

**14. Tag and push an image**

```
docker login
docker tag myapp:1.0 YOUR_USERNAME/myapp:1.0
docker push YOUR_USERNAME/myapp:1.0
```

Replace YOUR_USERNAME with your actual Docker Hub username.

**15. Delete images**

```
docker rmi myapp:1.0
docker image prune
```

**16. Volume commands**

```
docker volume ls
docker volume create mydata
docker volume inspect mydata
docker volume rm mydata
```

Example using a volume:

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=labpass \
  -v mydata:/var/lib/postgresql/data \
  postgres:16-alpine
```

**Part 2 – Kubernetes commands and scenario-based deployment**

**1. Check Kubernetes status**

```
kubectl version --client
kubectl cluster-info
kubectl get nodes
```

**2. Create resources from YAML**

resources using a YAML file.

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

kubectl apply -f .
Apply an entire directory:
kubectl apply -f k8s/
**3. Create a Deployment directly**
kubectl create deployment myapp --image=nginx
**4. Expose an application**
kubectl expose deployment myapp --port=80 --type=NodePort
kubectl expose deployment myapp --port=80 --target-port=80 --type=ClusterIP
**5. List resources**
kubectl get pods
kubectl get deployments
kubectl get services
kubectl get configmaps
kubectl get secrets
kubectl get pvc
kubectl get all
**6. Get detailed information**
kubectl describe pod POD_NAME
kubectl describe deployment DEPLOYMENT_NAME
kubectl describe service SERVICE_NAME
**7. View logs**
kubectl logs POD_NAME
kubectl logs -f POD_NAME
kubectl logs deployment/myapp
kubectl logs POD_NAME --previous
**8. Execute commands inside a Pod**
kubectl exec -it POD_NAME -- sh
kubectl exec POD_NAME -- env
**9. Scale a Deployment**
kubectl scale deployment myapp --replicas=3
Verify:
kubectl get deployments
kubectl get pods
**10. Update an image**
kubectl set image deployment/myapp myapp=nginx:1.27
kubectl rollout status deployment/myapp
The container name (myapp) must match the container name inside the Deployment.
**11. Roll back a Deployment**
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp
**12. Delete resources**
kubectl delete -f deployment.yaml
kubectl delete -f k8s/
kubectl delete deployment myapp
kubectl delete service myapp
kubectl delete pod POD_NAME
**13. Validate YAML before applying**
kubectl apply --dry-run=client -f deployment.yaml
kubectl diff -f deployment.yaml
**14. Access an application locally**
kubectl port-forward service/myapp 8080:80
Now open http://localhost:8080.
**15. Check events and troubleshoot**
kubectl get events --sort-by=.metadata.creationTimestamp

```
kubectl get pods -o wide
kubectl describe pod POD_NAME
kubectl logs POD_NAME
```

**B. Kubernetes YAML templates to memorize**

**1. Pod**

```
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: nginx
      image: nginx:alpine
      ports:
        - containerPort: 80
```

**# Deploy:** kubectl apply -f pod.yaml

```
kubectl get pods
```

**2. Deployment**

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 2
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:alpine
          ports:
            - containerPort: 80
```

Save as deployment.yaml, then:

```
kubectl apply -f deployment.yaml
kubectl get deployments
kubectl get pods
```

**3. Service**

```
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp
  ports:
    - port: 80
      targetPort: 80
  type: ClusterIP
```

Here, app: myapp must match the labels on the Deployment's Pods.